Function Overview
Core Libraries
Key Functions

Uses RTC to get real-time clock data and calculates the sun's azimuth based on geographic coordinates. The target angle is computed using a solar position algorithm. Then, LDR sensors are used for fine-tuning the final orientation to ensure accurate solar tracking.

RTClib.h → Reads real-time clock.
SunPosition.h → Computes solar azimuth using SPA (Solar Position Algorithm).
math.h → Provides basic math functions.

SunPosition() →Calculates the sunrise and sunset times of the day based on the time zone and geographic coordinates, thereby determining the full daytime interval.

computeLinearTgt() → Uses linear interpolation to map the current time's progress within the daytime interval to the motor's rotation range[−90°, +90°], and outputs the target azimuth

```
// === Linear interpolation to calculate the target angle ===
float computeLinearTgt(SunPosition& sunNow, const DateTime& nowTime,
                       int eastLimit, int westLimit) {
  float sunEl = sunNow.altitude();
  uint16_t sr = sunNow.sunrise();
  uint16_t ss = sunNow.sunset();
  uint16_t nowMin = nowTime.hour()*60 + nowTime.minute();          //get real time
  if (sunEl <= EL_NIGHT || ss <= sr || nowMin < sr || nowMin > ss)    //Return east at night
    return -eastLimit;
  float ratio = (float)(nowMin - sr) / (float)(ss - sr);            //Linear interpolation
  ratio = constrain(ratio, 0.0f, 1.0f);
  return -90.0f + ratio * 180.0f;
}
```

ldrDeltaSuggest() → Compares the readings from two LDR sensors (east and west) to

```
int ldrDeltaSuggest(bool &goWest, int &eRaw, int &wRaw) {
  eRaw = readLdrAvgRaw(LDR_EAST_PIN);
  wRaw = readLdrAvgRaw(LDR_WEST_PIN);
  int eB = LDR_LOW_IS_BRIGHT ? (1023 - eRaw) : eRaw;
  int wB = LDR_LOW_IS_BRIGHT ? (1023 - wRaw) : wRaw;
  goWest = (wB > eB);                          //Decide on the direction of fine-tuning
  if (LDR_FLIP_DIR) goWest = !goWest;
  return abs(wB - eB);                         //Returns the absolute value of the difference
}
```

loop for LDR() → Fine-tuning loop: within the set time, repeatedly calls ldrDeltaSuggest() to

```
uint32_t t0 = millis();
float cur = getAngle();
while (millis() - t0 < TRACK_TIME_BUDGET_MS) {              //Fine-tune within the set time
  bool goW; int er, wr;
  if (ldrDeltaSuggest(goW, er, wr) <= LDR_EXIT_THRESH) break;  //Call ldrDeltaSuggest()
  float next = cur + (goW ? +LDR_STEP_DEG : -LDR_STEP_DEG);
  next = constrain(next, -EAST_LIMIT, WEST_LIMIT);            //Calculate a new target angle
  if (!moveToAngle(next)) break;                              //The motor turns to the new angle
  delay(LDR_SETTLE_MS);
  cur = getAngle();
}
```

moveToAngle() →Moves the motor to the target angle.